IT PROFESSIONAL PORTFOLIO

Cloud Engineering and Security Projects

PROJECT 39

Explore EC2 User Data Secret Exposure

Amazon EC2 | User Data | IMDSv2 | Session Manager | CloudTrail

AUTHOR	Michael Salazar

ENVIRONMENT	Personal AWS Lab
VERSION	v1.0.0
RELEASE DATE	July 29, 2026
STATUS	Complete

Executive Summary

Created a new **SLAW** CloudFormation stack, launched an **Amazon Linux** EC2 instance, and intentionally placed a dummy username and password in the instance **user data** field to demonstrate why user data should not be used for secrets. The instance was launched with no key pair, the default security group, and the **SSMClient** IAM role for Session Manager access. After connecting to the instance, an IMDSv2 token was requested and used to retrieve **/latest/user-data** from the Instance Metadata Service. The user data was exposed from inside the instance, while the CloudTrail RunInstances event showed that AWS removed the sensitive user data from logs using **<sensitiveDataRemoved>**. The instance and stack were deleted after validation.

SECURITY OUTCOME	**User data is not a secrets-management system. It can be retrieved from inside the instance and by users with sufficient EC2 read permissions, even though CloudTrail scrubs it from event logs.**

Business Problem

User data is useful for bootstrapping instances and passing startup configuration, but it is often abused as a convenient place to store credentials. This project shows the practical exposure path: a logged-in user or attacker with instance access can retrieve the user data from the metadata service. It also shows the detection gap: CloudTrail protects operational logs by replacing the raw user data with a placeholder, which means responders may need direct EC2 access to inspect what was actually exposed before the instance is terminated.

Objectives

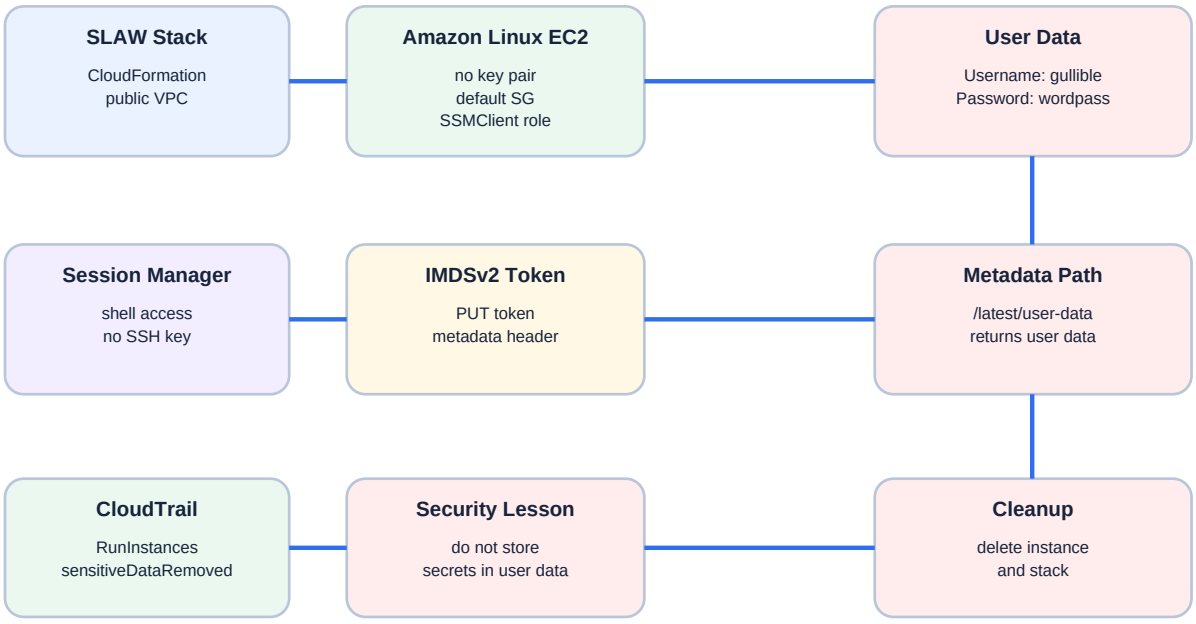
* Create the SLAW CloudFormation stack from the provided template.
* Launch an Amazon Linux instance named for the lab with no key pair assigned.
* Attach the instance to the default security group.
* Assign the SSMClient IAM role/instance profile so Session Manager can connect.
* Add dummy secret-like user data: Username gullible and Password wordpass.
* Connect to the instance using Session Manager.
* Use IMDSv2 token-based metadata access to retrieve /latest/user-data.
* Validate that the user data is exposed through the metadata service.
* Review the instance creation log/CloudTrail event and confirm userData was replaced with sensitiveDataRemoved.
* Terminate the EC2 instance and delete the CloudFormation stack.

Environment

Component	Configuration
Cloud Provider	Amazon Web Services (AWS)

Component	Configuration
Primary Services	Amazon EC2, EC2 User Data, Instance Metadata Service, AWS Systems Manager Session Manager, IAM, CloudFormation, CloudTrail
Account Context	TestAccount1 / AdministratorAccess
Stack Name	SLAW
Template Source	Provided public VPC CloudFormation template
Instance OS	Amazon Linux
Key Pair	None / proceed without a key pair
Security Group	Default security group
IAM Role / Instance Profile	SSMClient
User Data	Username: gullible; Password: wordpass
Metadata Access	IMDSv2 token then /latest/user-data
CloudTrail Observation	userData replaced with
Cleanup	Terminated instance and deleted CloudFormation stack

Architecture



User data is visible from EC2 read paths and from inside the instance through metadata; CloudTrail redacts it.

Figure 1 - User data exposure path through EC2 metadata and CloudTrail log redaction.

Implementation Summary

1. Signed into TestAccount1 with AdministratorAccess.

2. Created the SLAW CloudFormation stack from the provided public VPC template.

3. Launched a new Amazon Linux EC2 instance.

4. Proceeded without assigning a key pair.

5. Selected the default security group.

6. Attached the SSMClient IAM role/instance profile to the EC2 instance.

7. Entered dummy secret-like user data containing Username gullible and Password wordpass.

8. Connected to the instance using Systems Manager Session Manager.

9. Requested an IMDSv2 token using curl against the metadata token endpoint.

10. Used the token header to retrieve the user data from http://169.254.169.254/latest/user-data.

11. Reviewed the instance creation event/log output and confirmed the userData value was replaced by .

12. Terminated the EC2 instance and deleted the SLAW CloudFormation stack after the lab.

Command Pattern

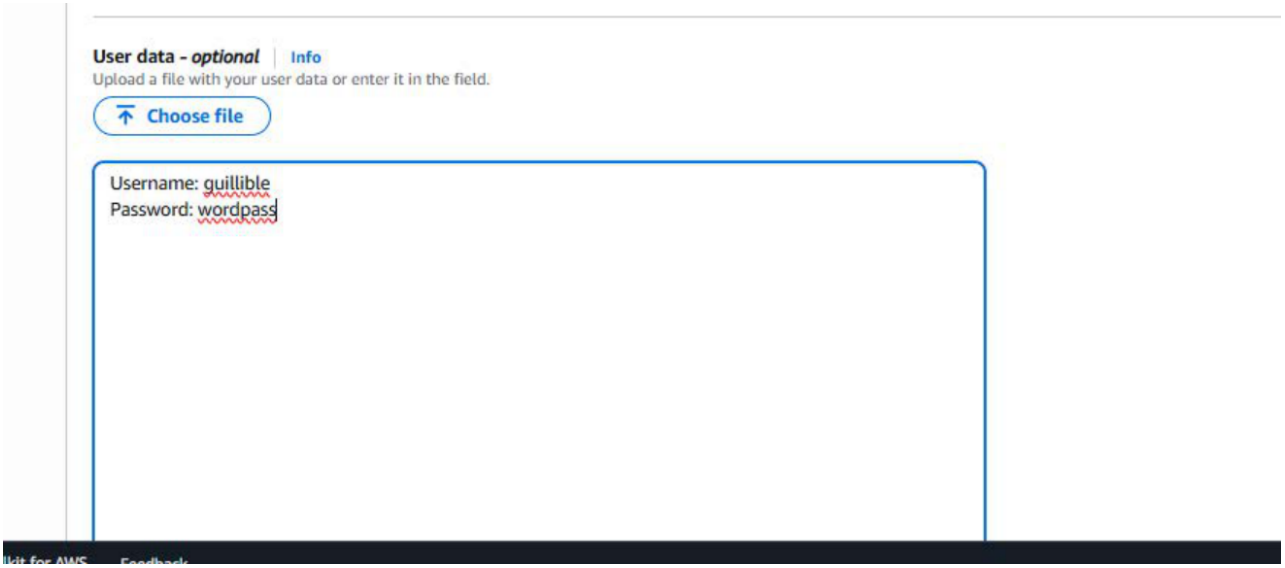
```
TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"`

curl -H "X-aws-ec2-metadata-token: $TOKEN" -v \
  http://169.254.169.254/latest/user-data
```

This IMDSv2 pattern retrieves an instance metadata token first, then sends the token as a header when requesting the user data path.

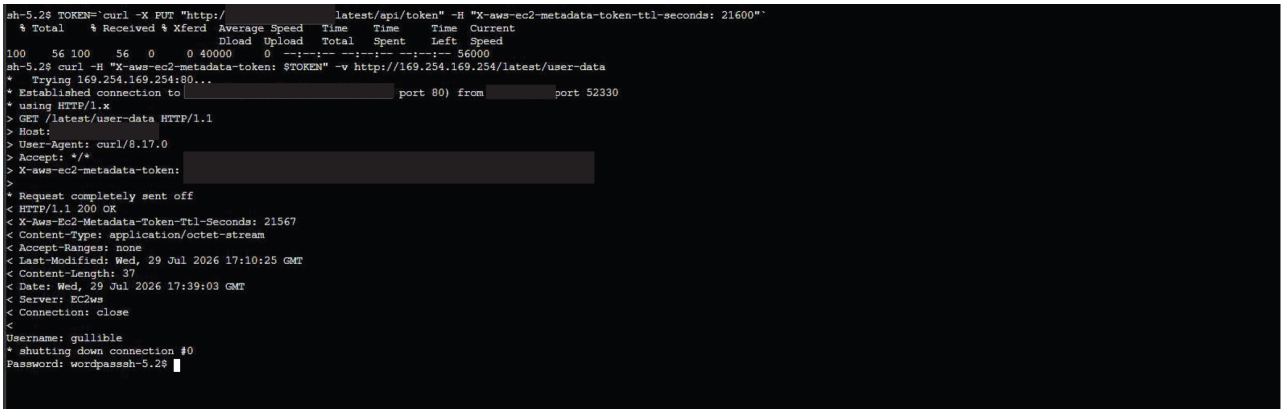
Evidence

Figure 2 - Dummy Secret-Like User Data Added at Launch



The launch configuration evidence shows the EC2 user data field populated with dummy secret-like values: Username gullible and Password wordpass. These values are intentionally non-production lab data.

Figure 3 - User Data Retrieved Through IMDSv2 Metadata



The metadata extraction evidence shows an IMDSv2 token request followed by a metadata request to /latest/user-data. The response returns the dummy username and password from the instance user data field.

Figure 4 - CloudTrail / Instance Log Redacts User Data

The log evidence shows userData represented as , confirming the raw user data was not preserved directly in the visible CloudTrail-style event output.

Security Analysis

User data is useful for initialization and automation, but it is not a protected secret store.

Any principal or process that can log into the instance can query the metadata service and retrieve the user data if it is still associated with the instance.

Users with sufficient EC2 read permissions may also be able to view user data from the EC2 console or API path.

CloudTrail redaction is a protective log-safety control, not a detection control. It prevents sensitive user data from spreading into logs, but it also means incident responders may not see the original secret in CloudTrail.

The correct fix is to avoid placing secrets in user data. Use a secrets manager, parameter store, short-lived credentials, and least-privilege instance roles instead.

The cleanup step matters because user data is only inspectable while the instance exists. Once the instance is terminated, the responder may lose direct access to the exposed content.

Lessons Learned

EC2 user data can pass configuration or scripts to an instance at launch.

Credentials, passwords, tokens, and API keys should never be placed in user data.

IMDSv2 still allows authorized metadata access from inside the instance; its token requirement does not make user data secret.

CloudTrail protects logs by replacing raw user data with .

Responders need to understand both exposure paths and evidence gaps when investigating user-data misuse.

Production Improvements

* Use AWS Secrets Manager or AWS Systems Manager Parameter Store for secrets.
* Use IAM roles and short-lived credentials instead of embedded passwords or static keys.
* Restrict EC2 permissions that allow users to view or modify user data.
* Require IMDSv2 on all instances, but do not treat IMDSv2 as a secrets-control replacement.
* Scan user data in IaC templates and launch workflows for secret patterns before deployment.
* Monitor RunInstances and ModifyInstanceAttribute events for user data changes.
* Use CloudTrail, EventBridge, AWS Config, and Security Hub controls to detect risky instance configuration.
* Build incident-response playbooks that preserve instance metadata and user data before termination when needed.

Skills Demonstrated

[illegible]	[illegible]	[illegible]
Amazon EC2	EC2 User Data	IMDSv2
Session Manager	CloudTrail review	CloudFormation cleanup
Secrets risk analysis	Incident-response nuance	Evidence handling

Resume Bullet

RESUME	Demonstrated EC2 user data secret exposure by launching an Amazon Linux instance with dummy credentials in user data, retrieving the values through IMDSv2 metadata access, validating CloudTrail redaction as , and documenting safer secrets-management controls.

STAR Interview Story

Situation: The portfolio had already explored access keys and metadata-service credential risks. The next gap was how user data can accidentally expose secrets.

Task: Show where user data is visible, how it can be retrieved from an instance, and how AWS handles it in logs.

Action: I created the SLAW stack, launched an Amazon Linux instance with no key pair, the default security group, and the SSMClient role, added dummy username/password values to user data, connected through Session Manager, retrieved /latest/user-data through IMDSv2, confirmed CloudTrail-style output showed , and deleted the instance and stack.

Result: The lab proved that user data is not safe for secrets, even though CloudTrail redacts it, and documented production controls for proper secrets management.

Version History

Version	Release Date	Description
v1.0.0	July 29, 2026	Initial documented implementation. Created SLAW stack, launched Amazon Linux instance, assigned default security group and SSMClient role, added dummy secret-like user data, retrieved user data through IMDSv2 metadata access, confirmed CloudTrail userData redaction, deleted instance and stack, and embedded provided evidence using portfolio documentation standard v1.0.1.

References

* Cloud Security Lab a Week (S.L.A.W.) - Explore the Power and Pain of User-Data.

* User-provided EC2 user data evidence screenshot.

* User-provided metadata extraction evidence screenshot.

* User-provided CloudTrail userData redaction evidence screenshot.

* Project 38 - Compare IMDSv1 and IMDSv2 Metadata Credential Risk.